HITL Sample Size Planner — Specification v1.0

Status	v1.0 — all decisions resolved in the spec interview of 27 Sep 2026 (Q1–Q22, Appendix A)
Owner	TJ (sole user, sole maintainer)
Build family	Same pattern as the Pareto Chart Tool and IMR Control Chart Tool: one Python file, local Flask server, browser as the screen
Build tool	Claude Code, blueprint-driven (`/step N`), one step per session, commit after each green step
Method version	1.0 (changes to any formula, constant or answer-key value bump this)
File	`hitl_planner.py` (tool) · `test_hitl_planner.py` (tests) · `CLAUDE.md` (guardrails, §12)

How to read this document. "MUST" is a requirement. Every default the interview settled is stated explicitly; the few defaults assumed without an explicit question are listed in Appendix B so they can be overturned. §4 (calculations) and §11.1 (answer key) are the contract: if the code and the answer key disagree, the code is wrong.

---

1. Purpose and scope

1.1 What it does (plain English)

An AI governance lead needs subject-matter experts (SMEs) to manually review a sample of AI-generated conversations each period, so the resulting report can make a defensible statement about how often the AI gets things wrong. Too small a sample and the statement is meaningless; too large and the SMEs drown.

The tool answers, for one AI system and one period:

1. **How many conversations must be reviewed** to state the issue rate to a chosen precision, and (optionally) to certify that the rate is below a threshold.
2. **How many SME hours that costs**, including a second independent review of a subset (needed to measure reviewer consistency), and whether it **fits** the SME capacity available.
3. **If it doesn't fit, what the available hours buy** — the best precision and the lowest certifiable threshold — with a slider to explore the trade-off.
4. **What the resulting report can and can't claim**, in plain English, plus every formula and every intermediate number so an auditor can reproduce the headline with a calculator.
5. After the review is done, **what was actually achieved** (observed rate, achieved margin, reviewer agreement), which feeds the next cycle's plan.

Every plan is saved as an immutable record; a generated CSV log gives a cycle-over-cycle view across systems.

1.2 Who and where

Personal desktop tool for TJ's own use. Runs on his Windows machine, bound to `127.0.0.1` only. No login, no network calls, no telemetry. It never sees conversation content — only counts and rates.

1.3 In scope (v1)

Plan tab, Actuals tab, History tab, Method tab; the calculations in §4; warnings in §5; statements in §6; persistence in §8; tests in §11.

1.4 Out of scope (parked for v2 — do not build, do not "helpfully" add)

- Strata with Neyman allocation
- Uploading a conversation log to draw the actual random sample (needs a data-handling review before anything is uploaded)
- Generated PDF (e.g. reportlab); PyInstaller executable
- Cohen's kappa computed from double-review data; a confidence interval on the observed rate
- Trend view across cycles per system; SME cost in currency

- Deleting plans from the UI

---

2. Architecture

2.1 Shape

- **One file,** `hitl_planner.py`, containing: constants, calculation functions, validation, persistence, the Flask app, and the HTML/CSS/JS of the page as string templates. Nothing is loaded from the internet at any time (no CDN fonts, no CDN scripts); the tool MUST work fully offline.
- **All maths in Python.** The browser never computes a statistic. A small JavaScript layer sends the current inputs to `POST /api/calc` (debounced, ~150 ms after the last keystroke) and paints the returned results. No JS framework.
- **Flask** serves the page and a JSON API (§9). **Python 3.11.** Dependencies: Flask only; everything else from the standard library (`math`, `json`, `csv`, `pathlib`, `datetime`, `webbrowser`, `os`, `re`, `argparse`, `socket`).
- **Desktop only.** No mobile layout.
- **Look and feel:** the same visual conventions as the Pareto and IMR tools (system font stack, light theme, card layout, inline CSS). Reuse their CSS where it exists.

2.2 Files on disk

Repository (next to the script):

```
hitl_planner.py          the tool (single source of truth)
test_hitl_planner.py     pytest suite (§11)
CLAUDE.md                non-negotiables for every Claude Code
session (§12)
config.json              created on first run (§8.6); not
committed
README.md                how to run, how to test (short)
```

Workspace (user data, default `%USERPROFILE%\Documents\HITL Planner\`, §8.1):

```
plans_log.csv                          generated view, never
hand-edited
```

```
2026-10 - HR Helpdesk Bot\
    plan_v1.json                        immutable
    plan_v2.json                        immutable
    actuals_v1.json                     immutable
    report.html                         regenerated on every save
2026-W41 - Recruiting Assistant\
    ...
```

2.3 Running it

1. `python hitl_planner.py` — reads `config.json` (creating it with defaults if absent), runs the startup integrity check (§7.5), starts Flask on `127.0.0.1:<port>`, opens the default browser at `http://127.0.0.1:<port>/`.
2. **Fixed port**, default `5062`, stored in `config.json`. The build MUST check the Pareto and IMR scripts and pick a port that does not collide with either; record the chosen value in Appendix B on first commit.
3. **Single instance.** Before binding, `GET http://127.0.0.1:<port>/api/health` is attempted. If it answers with this tool's name, the script opens the browser to the running instance and exits 0. If the port is held by something else, the script prints a one-line message naming the port and `config.json`, and exits 2.
4. **Quit button** on every tab: `POST /api/quit` shuts the server down cleanly. Ctrl-C in the terminal also works.
5. `python hitl_planner.py --selftest` runs the 11 answer-key cases (§11.1), prints PASS/FAIL per case, and exits 0/1 without starting the server.

2.4 Data stance

Bound to `127.0.0.1` only. No outbound HTTP. No analytics. Inputs are counts, rates, hours and free-text notes; conversation content is never entered, stored or requested.

---

3. Domain model

3.1 Plan identity and versions

Concept	Definition
Period type	`month` or `week`. Chosen per plan. No custom date ranges.
Period label	Month: `YYYY-MM` (e.g. `2026-10`). Week: ISO week `YYYY-Www` (e.g. `2026-W41`). Display forms: `Oct 2026`, `2026-W41`.
System / use case	Free text, 1–60 characters, e.g. `HR Helpdesk Bot`. Auto-complete from systems already saved. Matching for prefill and auto-complete is case-insensitive and whitespace-trimmed on the `system` field stored in the JSON, not on folder names.
plan_id	`<period label> - <sanitised system>` with a plain hyphen, date-first so folders sort chronologically: `2026-10 - HR Helpdesk Bot`. This is the sub-folder name.
Plan version	Saving into an existing `plan_id` folder creates `plan_v<N+1>.json`; earlier versions are never modified. "Latest" = highest N.
Actuals version	`actuals_v<N>.json`, same rule. Actuals reference the plan version they were recorded against.
"Most recent plan for a system"	The plan JSON with the latest `saved_at` among all folders whose `system` matches. Period type is irrelevant to this lookup.
"Most recent actuals for a system"	The actuals JSON with the latest `recorded_at` among matching folders.

Folder-name sanitising (`sanitise_system`): remove `< > : " / \ | ? *` and ASCII control characters; collapse runs of whitespace to one space; strip leading/trailing spaces and dots; truncate to 60 characters; if the result is a reserved Windows name (`CON, PRN, AUX, NUL, COM1–COM9, LPT1–LPT9`, case-insensitive) append `_`; if the result is empty, it is a validation error. If a folder with that name already exists whose latest plan has a *different* `system` (case-insensitive), append `(2)`, `(3)`, … to the folder name.

3.2 Inputs

Symbols in the third column are used in §4.

#	Field	Symbol	Type and limits	Default, new system	Prefill, existing system
I1	Period type	—	`month` / `week`	`month`	last plan's
I2	Period	—	picker only; `YYYY-MM` or `YYYY-Www`	current month / ISO week (local date)	current — never carried over
I3	System / use case	—	text, 1–60 chars, auto-complete	blank, **required**	as chosen by the user
I4	Population N	N	integer ≥ 1	blank, **required**	blank, **required** (changes every cycle)
I5	Confidence level	C	one of 90 / 95 / 99 %	95 %	last plan's
I6	Margin mode	—	`absolute` / `relative`	`absolute`	last plan's
I7	Margin of error	E (abs) or r (rel)	absolute 0.5–25 %; relative 5–50 %	±5 % absolute	last plan's
I8	Expected issue rate	p	0.1–99.9 %	50 %	last observed rate for the system, floored at 0.1 %; if no actuals

#	Field	Symbol	Type and limits	Default, new system	Prefill, existing system
					exist, last planned p
I9	Number of SMEs	S	integer ≥ 1	blank, **required**	last plan's
I10	Hours per SME per period	H	> 0, decimals allowed	blank, **required**	last plan's
I11	Minutes per conversation review	M	> 0, decimals allowed	blank, **required**	last plan's
I12	Double-review share	s	0–100 %	15 %	last plan's
I13	Double-review floor	F	integer ≥ 0	50	last plan's
I14	Certify toggle	—	on / off	on	last plan's
I15	Certify threshold	T	0.01–50 %	1 %	last plan's

Rules:

- Values outside the limits are **refused with a field-level message**, never silently clamped (§10 E1).
- The principle behind the defaults: statistical conventions are preset; facts about the team (N, S, H, M) are never guessed.
- **Percent handling.** The UI shows and accepts percentages (`5` , `0.5`); the API and JSON store decimals (`0.05` , `0.005`). All formulas in §4 use decimals.

Tooltips (one line each, shown on hover/focus):

- I4 — "Total conversations the AI produced in this period. The sample is drawn from these."

- I5 — "How sure you want to be that the true rate lies inside the margin. 95% is the convention."
- I6/I7 — "Absolute: ±5 percentage points. Relative: ±25% of the rate itself — use this when the expected rate is small."
- I8 — "Your best guess of the issue rate. Leave at 50% if unknown — that gives the largest (safest) sample."
- I9–I11 — "Net review time actually available this period; exclude calibration meetings and admin."
- I12/I13 — "Share of the sample reviewed independently by a second SME, so reviewer agreement can be measured. Never fewer than the floor, never more than the sample."
- I14/I15 — "Also size the sample so that, if no issues are found, you can state the rate is below this threshold."

3.3 Prefill and `p_source`

When the System field matches a saved system, the tool calls `GET /api/prefill?system=…` and fills I1, I5–I7, I9–I15 from the most recent plan, and I8 per the rule above. N stays blank. A banner reads: "**Prefilled from 2026-09 - HR Helpdesk Bot (v2) · p from actuals recorded 2026-10-03**" (second clause only when p came from actuals). Any field the user then edits is kept as edited.

`p_source` is recorded with every plan: `default` (the 50 % preset, untouched), `manual` (typed by the user), `last_observed` (from actuals), `last_planned` (carried from the previous plan). Warning W2 (§5) keys on the presence of an observed rate, not on `p_source`.

---

4. Calculations (the contract)

All inputs as decimals. `ln` is the natural logarithm. **Every n is rounded UP to a whole number.**

4.1 Constants — exact, not to be "improved"

```
Z = {0.90: 1.645, 0.95: 1.960, 0.99: 2.576}
```

These rounded values are part of the contract. With z = 1.95996 the first answer-key case gives 384 instead of 385. The build MUST use exactly these three numbers.

4.2 Helper functions

```
ceil_safe(x)   = ceil(round(x, 9))        # absorbs floating-
point noise before rounding up
fits(hours, capacity) = hours <= capacity + 1e-9
```

4.3 Formulas

#	Name	Formula	Notes
F1	Effective absolute margin	absolute mode: `E = E_abs` ; relative mode: `E = r × p`	E is what every other formula uses
F2	Base sample (Cochran)	`n0 = Z² × p × (1 − p) / E²`	
F3	Finite population correction	`n_cochran = ceil_safe( n0 / (1 + (n0 − 1) / N) )`	For N = 1 the expression gives 1
F4	Assurance sample (rule of three, exact form)	`n_assure = ceil_safe( ln(1 − C) / ln(1 − T) )`	"Review n_assure; if none shows the issue, the rate is below T with confidence C"
F5	Headline sample	certify on and `n_assure ≤ N` : `n = max(n_cochran, n_assure)` ; otherwise `n = n_cochran`	`binding = "assurance"` if certify on, `n_assure ≤ N` and `n_assure > n_cochran` ; else `"cochran"` . If certify on and `n_assure > N` , `binding = "cochran"` and W7 fires

#	Name	Formula	Notes
F6	Double-review count	`DR = min(n, max(F, ceil_safe(s × n)))`	Never more than n
F7	Reviews, hours, capacity, verdict	`reviews = n + DR`; `hours = reviews × M / 60`; `capacity = S × H`; `verdict = FITS if fits(hours, capacity) else DOES NOT FIT`	Verdict uses exact arithmetic; display rounds
F8	Detection power	`P = 1 − (1 − T) ^ n`	Evaluated at the headline n and the certify threshold, whether or not certify is on
F9	Reverse mode (margin at a given n)	`E_at(n) = Z × sqrt( p(1−p)/n × (N − n)/(N − 1) )`; if `n ≥ N`: `E_at = 0`	Relative form: `E_at(n) / p`
F10	Certify-backwards (threshold at a given n)	`T_at(n) = 1 − (1 − C) ^ (1 / n)`	Inverse of F4; undefined for n = 0 (show "n/a")

4.4 Trade-off panel (shown only when verdict is DOES NOT FIT)

1. **Capacity-max sample.** `n_max` = the largest integer n in `[0, min(N, floor(capacity × 60 / M + 1e-9))]` such that `fits(hours(n), capacity)` where `hours(n) = (n + min(n, max(F, ceil_safe(s × n)))) × M / 60`. `reviews(n)` is non-decreasing in n, so a binary search is correct; a downward linear scan is acceptable. Both the panel and the headline MUST use the same `hours()` / `fits()` code.
2. **What n_max buys:** `E_at(n_max)` (and its relative form in relative mode) via F9; `T_at(n_max)` via F10; `DR`, reviews and hours at n_max. If `n_max = 0`, the panel says capacity allows no reviews at all and shows no margin.
3. **What-if slider.** Range ±2 % to ±15 % absolute (step 0.5 %) or ±5 % to ±50 % relative (step 1 %), initial position = the current margin. Moving it

recomputes n, DR, hours and verdict live *for display only*, using the headline rule F5 (so with certify on, n cannot fall below `n_assure`, and the read-out says so) — the plan's inputs do not change. "**Adopt this margin**" writes the slider value into I7; the panel then disappears if the plan now fits. Confidence is not on the slider.

4.5 Sensitivity grid (always shown)

3 × 3 cells: margins {3 %, 5 %, 10 %} absolute or {10 %, 25 %, 50 %} relative (matching the current mode) × confidence {90 %, 95 %, 99 %}. Each cell shows n and SME hours computed with all other inputs as currently set, **using the headline rule F5** (so with certify on, a cell shows the larger of Cochran and assurance for that confidence, with a small "A" mark when assurance binds). Cells are coloured green (fits) or red (does not fit). The cell matching the current confidence and margin is highlighted; if the current margin is a custom value not in the grid, nothing is highlighted.

4.6 Actuals calculations

Inputs: `reviewed` (1…N), `issues` (0…reviewed), `agreement` (0–100 %), `notes` (free text). `n_planned`, `E_planned`, `p_planned`, `T`, `C`, `N` and the certify flag come from the plan version referenced.

Output	Rule
Observed rate	`obs = issues / reviewed`
Achieved margin	if `0 < obs < 1`: `E_at(reviewed)` from F9 with `p = obs` (relative form also shown in relative mode). If `issues = 0`: no margin; instead "no issues found — with C confidence the true rate is below `T_at(reviewed)`" (F10). If `issues = reviewed`: "every reviewed conversation showed the issue — with C confidence the true rate is above `1 − T_at(reviewed)`"
Certify claim status	certify off → `not planned`. Certify on and `issues > 0` → `not applicable (issues found; report the observed rate)`. Certify on and `issues = 0` → `met` if `T_at(reviewed) ≤ T`, else `not met — can certify below T_at(reviewed) only`

Output	Rule
Planned vs actual	table: n planned vs reviewed; margin planned vs achieved; certify claim; DR planned vs (agreement entered)
Next-cycle p	`max(obs, 0.001)` becomes the I8 prefill for this system

4.7 Rounding and display

- Every n, `n_max` and DR: whole numbers, rounded up (`ceil_safe`). Thousands separators on n and N.
- Hours: 1 decimal. Capacity: 1 decimal.
- Percentages (margins, rates, thresholds, detection power, agreement): 1 decimal; **rates and thresholds below 1 % show 2 decimals** (e.g. `0.25 %`).
- The verdict and the trade-off search use unrounded values.

4.8 Edge cases the code MUST handle

Case	Behaviour
N = 1	F3 gives n = 1; F9 returns 0 (census)
n ≥ N in any formula	F9 returns 0; W6 fires; headline still shows n (capped at N by F3's construction)
certify on, `n_assure > N`	headline uses `n_cochran` ; W7 (red) fires; S2 uses its infeasible wording; plan can still be saved
`n_max = 0`	panel wording per §4.4; no F9/F10 values
`issues = 0` or `issues = reviewed`	§4.6 bound statements instead of a margin
p = 0.5 exactly, no history	W2 fires
Custom margin not in grid	no highlight

Case	Behaviour
Two browser tabs open	every calculation is stateless; the version number is determined by scanning the folder at the moment of saving

---

5. Warnings

Yellow = advisory, plan can be saved. Red = the requested claim cannot be made; the plan can still be saved and the red warning is recorded in the JSON. Warnings appear as banners at the top of the outputs column (Plan) or above the results (Actuals), and are stored by code in `plan_vN.json` / `actuals_vN.json` and in the log.

Code	Tab	Severity	Trigger	Message (values substituted)
W1	Plan	yellow	effective absolute margin `E > 0.10` (either mode)	"Margin of error is above ±10%. The reported rate will be too imprecise for most governance decisions — tighten the margin or add SME hours."
W2	Plan	yellow	`p = 0.50` and no observed rate exists for this system	"Expected issue rate is at the 50% default. This gives the largest sample; enter last cycle's rate if you have one."
W3	Plan	yellow	absolute mode and `E ≥ p`	"The margin (±{E}) is as large as the expected rate ({p}): the result could not tell the rate apart from zero. Switch to a relative margin, or rely on the certify claim."
W4	Plan	yellow	`DR < F` (the count had to	"The sample ({n}) is smaller than the double-

Code	Tab	Severity	Trigger	Message (values substituted)
			be capped at n)	review floor ({F}); every conversation will be double-reviewed and agreement will rest on fewer than {F} items."
W5	Plan	yellow	`N < 200`	"Population is under 200 — consider reviewing every conversation instead of sampling."
W6	Plan	yellow	`n > 0.8 × N`	"The sample is more than 80% of the population; review everything — sampling saves little here."
W7	Plan	**red**	certify on and `n_assure > N`	"Cannot certify that the rate is below {T} at this population: it would need {n_assure} clean reviews and only {N} conversations exist. Review everything and report the observed rate."
W8	Actuals	yellow	`reviewed < n_planned`	"Fewer conversations were reviewed ({reviewed}) than planned ({n_planned}). The achieved margin is ± {E_achieved}, not the planned ±{E_planned}."
W9	Actuals	yellow	`obs > 2 × p_planned` or `obs < 0.5 × p_planned`	"The observed rate ({obs}) is far from the planned expectation ({p_planned}). Next cycle's p will move; treat this cycle's precision claim with care."

Code	Tab	Severity	Trigger	Message (values substituted)
W10	Actuals	yellow	`agreement < 0.80`	"Double-review agreement is {agreement}, below 80%. Reviewer consistency is low — calibrate before relying on the rate."

W5 and W6 may fire together. Each warning MUST have a boundary test (§11.2).

---

6. "What this sample can and can't say" — statement templates

Always seven sentences, generated from the current numbers, shown on the Plan tab and copied into `plan_vN.json` and `report.html`. `{period}` is the display form (`Oct 2026` / `2026-W41`).

Can say

- S1 (absolute): "The reported issue rate will be within ±{E} of the true rate for {system} in {period}, with {C} confidence."
  S1 (relative): "The reported issue rate will be within ±{r} of its own value (± {E} absolute at the expected rate of {p}), with {C} confidence."
- S2 (certify on, feasible): "If none of the {n} reviewed conversations shows the issue, you can state with {C} confidence that the true rate is below {T}."
  S2 (certify on, W7): "The 'below {T}' claim cannot be made at this population: it would need {n_assure} clean reviews and only {N} conversations exist."
  S2 (certify off): "This plan is not sized to certify that the rate is below a threshold; a 'below {T}' claim would need {n_assure} clean reviews."
- S3: "If the true rate were {T}, this sample has a {P} chance of containing at least one such conversation."

Can't say

- **S4**: "It says nothing about conversations outside {period}, or about systems other than {system}."
- **S5**: "The ±{E} applies to the overall rate only — not to any team, region or question type. Sub-groups need their own sample."
- **S6**: "These claims assume the {n} conversations are picked at random from all {N}. Hand-picking, or reviewing only flagged conversations, voids them."
- **S7**: "It measures how often SMEs find issues, not how severe they are or why they happen."

---

7. Screens

Four tabs in a top bar, plus a **Quit** button at the far right. A banner strip below the tab bar shows global notices (integrity check failure, workspace not writable). The page title is "HITL Sample Size Planner".

7.1 Plan tab

Left column — inputs, in four groups with a heading each:

1. *Identity*: period type (segmented control), period picker, system (text with datalist auto-complete). Prefill banner appears under this group.
2. *Population and precision*: N; confidence (dropdown); margin mode (segmented) with margin value (dropdown 3 / 5 / 10 in absolute mode, 10 / 25 / 50 in relative mode, plus a "custom" entry that reveals a number field); expected issue rate p, with the one-line note "Leave at 50% if unknown — this gives the safest (largest) sample" and a small tag showing `p_source`.
3. *SME capacity*: S, H (label reads "hours per SME this month/week" following the period type), M.
4. *Double review and certify*: s, F, certify toggle, T.

Required-and-blank fields are outlined; while any input is invalid the outputs column is greyed with the message "Complete the highlighted fields" and Save is disabled.

Right column — outputs, top to bottom:

1. Warning banners (§5), if any.

2. **Headline card**: "Review **{n:,}** conversations = **{hours}** SME hours in {period}" with a green **FITS** / red **DOES NOT FIT** badge; sub-line "{n} + {DR} double reviews = {reviews} reviews × {M} min · capacity {capacity} h ({S} SMEs × {H} h)"; a third line naming the binding requirement: "n set by the ±{E} precision target" or "n set by the 'below {T}' certify claim (precision alone would need {n_cochran})".
3. **Trade-off panel** (only when DOES NOT FIT), §4.4: "Your capacity allows **{n_max}** conversations ({n_max} + {DR} double = {hours} h). That buys ± {E_at} precision and lets you certify below {T_at}." Then the what-if slider with its live read-out and the "Adopt this margin" button.
4. **Sensitivity grid**, §4.5.
5. **Assurance card**: n_assure and the statement "Review {n_assure} conversations; if none shows the issue, you can state with {C} confidence the true rate is below {T}." Red styling when W7. Under it, one line: detection power (S3 wording).
6. **What this sample can and can't say** — S1–S7.
7. **Save plan** button. On success: toast "Saved as {plan_id} v{N}" with links "Open report" and "Open folder". After saving, further edits are allowed; saving again creates the next version.

7.2 Actuals tab

- Dropdown of saved plans (`plan_id` , latest version, verdict, saved-at), newest first. Selecting one shows a read-only summary: n planned, DR planned, margin, certify threshold, p planned, N.
- If actuals already exist, they are shown read-only with their version; "Record again" opens the form for a new version.
- Form: reviewed, issues, agreement (%), notes (multi-line). Validation per §3.2/§4.6.
- Results (live on valid input): observed rate; achieved margin or bound statement; certify claim status; planned-vs-actual table; warnings W8–W10.
- **Record actuals** button → `actuals_v<N>.json` , regenerate `report.html` , rebuild `plans_log.csv` , toast with "Open report".

7.3 History tab

- Table from the same data as the log (§8.5), newest period first: period, system, version, saved-at, n, hours, verdict, warnings (codes), actuals

present (✔/—), observed rate, achieved margin.

- Filter box by system (substring, case-insensitive).
- Per row: **Load** (fills the Plan tab with that plan's inputs, N included, and switches to it; period stays as saved), **Report** (`POST /api/open-report`), **Folder** (`POST /api/open-folder`).
- Rows for unreadable folders show "unreadable — {error}" and have no buttons.
- No delete.

7.4 Method tab

1. **Answer-key status**: "Answer key: 11 / 11 passed at startup" (red with the failing cases listed if not).
2. **Formulas** F1–F10 as displayed in §4.3, with symbols defined and the constants table.
3. **Assumptions** in plain language: conversations are selected at random from all N; each conversation is scored simply "issue / no issue"; p = 50 % is the worst case for sample size; the "below T" claim requires zero issues found in the sample; detection power ignores the finite-population adjustment, which is negligible when n is small against N; SME time is uniform across SMEs.
4. **This plan's arithmetic**: the current Plan-tab inputs pushed through F1–F8 step by step, showing each intermediate value before and after rounding (Z → E → n0 → corrected n → n_assure → headline and binding → DR → reviews → hours → capacity → verdict → detection). Updates live with the Plan tab.
5. **References**: Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). Wiley. — Hanley, J. A., & Lippman-Hand, A. (1983). "If nothing goes wrong, is everything all right? Interpreting zero numerators." *JAMA*, 249(13), 1743–1745. — This tool's answer key (§11.1).
6. **Method version 1.0 (27 Sep 2026)** and tool version.

7.5 Startup integrity check

At every start (and in `--selftest`) the tool runs the 11 answer-key cases against its own functions. All pass → nothing visible beyond the Method-tab line. Any failure → a red banner on every tab ("Answer key failed: cases {list}. Results cannot be trusted; saving is disabled."), Save and Record buttons disabled, details on the Method tab, and the failures printed to the console.

8. Persistence

8.1 Workspace

Path from `config.json` (§8.6); default `%USERPROFILE%\Documents\HITL Planner`. Created on first run. Every plan lives in `<workspace>\<plan_id>\`.

8.2 `plan_v<N>.json` (immutable)

Annotated example; all keys are required unless marked optional. Decimals for rates; ISO-8601 timestamps with the local UTC offset.


```
{
  "schema_version": 1,
  "tool": {"name": "HITL Sample Size Planner", "version":
"1.0.0", "method_version": "1.0"},
  "saved_at": "2026-10-03T09:14:22+05:30",
  "plan": {"plan_id": "2026-10 - HR Helpdesk Bot",
"period_type": "month", "period_label": "2026-10",
          "period_display": "Oct 2026", "system": "HR Helpdesk
Bot", "version": 1},
  "inputs": {
    "N": 50000, "confidence": 0.95, "margin_mode": "absolute",
"margin_input": 0.05,
    "p": 0.5, "p_source": "default", "smes": 3, "hours_per_sme":
8, "minutes_per_conversation": 12,
    "dr_share": 0.15, "dr_floor": 50, "certify": true,
"threshold": 0.01,
    "prefilled_from": null
  },
  "outputs": {
    "z": 1.960, "E_effective": 0.05, "n0": 384.16,
"n_cochran_raw": 381.24, "n_cochran": 382,
    "n_assure_raw": 298.07, "n_assure": 299, "n_headline": 382,
"binding": "cochran",
    "dr_count": 58, "reviews": 440, "hours": 88.0, "capacity":
24.0, "verdict": "DOES NOT FIT",
    "detection_power": 0.9785,
    "tradeoff": {"n_max": 70, "dr_count": 50, "reviews": 120,
"hours": 24.0,
                "margin_at_capacity": 0.1171,
"margin_at_capacity_relative": null, "threshold_at_capacity":
0.0419},
```

```
    "sensitivity": [
      {"confidence": 0.90, "margin": 0.03, "n": 741, "dr_count":
112, "hours": 170.6, "fits": false, "binding": "cochran"}
    ]
  },
  "warnings": [{"code": "W2", "severity": "yellow", "message":
"Expected issue rate is at the 50% default. ..."}],
  "statements": ["S1 text", "S2 text", "S3 text", "S4 text", "S5
text", "S6 text", "S7 text"],
  "method": {
    "constants": {"Z": {"0.90": 1.645, "0.95": 1.960, "0.99":
2.576}},
    "formulas": {"F1": "E = E_abs (absolute) or r × p
(relative)", "F2": "n0 = Z^2 p(1-p)/E^2", "...": "..."},
    "arithmetic": ["Z for 95% = 1.960", "E = 0.05", "n0 =
1.960^2 × 0.5 × 0.5 / 0.05^2 = 384.16", "..."]
  },
  "integrity_check": {"passed": 11, "total": 11}
}
```


(`tradeoff` is `null` when the plan fits. The `sensitivity` array has nine entries; one is shown. Illustrative numbers above are from the acceptance scenario in §11.6, not part of the answer key.)

8.3 `actuals_v<N>.json` (immutable)


```
{
  "schema_version": 1,
  "tool": {"name": "HITL Sample Size Planner", "version":
"1.0.0", "method_version": "1.0"},
  "recorded_at": "2026-11-02T18:40:05+05:30",
  "plan_id": "2026-10 - HR Helpdesk Bot", "plan_version": 1,
"version": 1,
  "inputs": {"reviewed": 382, "issues": 11, "agreement": 0.86,
"notes": "Two SMEs, calibration on 3 Oct."},
  "outputs": {"observed_rate": 0.0288, "achieved_margin":
0.0167, "achieved_margin_relative": null,
              "bound_statement": null, "certify_status": "not
applicable (issues found; report the observed rate)",
              "planned_vs_actual": {"n_planned": 382,
"reviewed": 382, "margin_planned": 0.05, "margin_achieved":
0.0167}},
  "warnings": [{"code": "W9", "severity": "yellow", "message":
```

```
"..."}]
}
```

8.4 `report.html` (regenerated on every save and every actuals entry)

Self-contained: inline CSS, no scripts, no external assets; A4 print stylesheet with sensible page breaks; header "HITL Sample Size Planner — Sampling plan" with plan_id, version, saved-at, tool and method version. Sections in order: inputs table; headline and verdict; trade-off (if any); sensitivity grid; assurance and detection; the seven statements; warnings; this plan's arithmetic; formulas and constants; references; **Actuals** (planned vs actual, observed rate, achieved margin or bound, certify status, agreement, notes, W8–W10) when actuals exist. "PDF" = open the report in the browser and print to PDF; the tool does not generate PDFs.

8.5 `plans_log.csv` (generated, never appended, never hand-edited)

Rebuilt from scratch from every readable folder after each save or actuals entry, and on `GET /api/log/rebuild`. One row per plan (latest plan version + latest actuals version). Encoding `utf-8-sig` so Excel opens it correctly; standard `csv` quoting (notes may contain commas and line breaks). Sorted by `period_label` descending, then `system` ascending.

Columns, in order:

`plan_id, period_type, period_label, system, plan_version, saved_at, N, confidence, margin_mode, margin_input, E_effective, p, p_source, certify, threshold, smes, hours_per_sme, minutes_per_conversation, dr_share, dr_floor, n_cochran, n_assure, n_headline, binding, dr_count, reviews, hours, capacity, verdict, n_max_at_capacity, margin_at_capacity, threshold_at_capacity, detection_power, warnings, actuals_version, recorded_at, reviewed, issues, observed_rate, achieved_margin, certify_status, agreement, notes`

`warnings` = codes joined by `;` (e.g. `W2;W6`). Actuals columns are empty until actuals exist. Unreadable folders are skipped with a console line.

8.6 `config.json` (next to the script)

```
{"schema_version": 1, "workspace":
```

```
"C:\\Users\\tj\\Documents\\HITL Planner", "port": 5062}
```

Created with defaults if missing. If present but unreadable or missing a key, the tool prints a one-line message and exits 2 (it never guesses over a broken config).

8.7 Write discipline

- **Atomic writes:** write to `<name>.tmp` in the same folder, then `os.replace` to the final name. On failure, delete the temp file and report (§10 E3). Applies to JSON, HTML, CSV and config.
- **Immutability:** the tool never opens `plan_v*.json` or `actuals_v*.json` for writing. The next version number is `max(existing) + 1`, determined by scanning the folder at the moment of saving.
- **Single source of truth:** JSON files are the record; `report.html` and `plans_log.csv` are derived and can be regenerated at any time.
- JSON written with `ensure_ascii=False`, indent 2, UTF-8.

---

9. API (Flask, JSON, all under `127.0.0.1`)

Method and path	Purpose	Request → Response
`GET /`	the single page	HTML
`GET /api/health`	single-instance check	`{"tool": "HITL Sample Size Planner", "version": "1.0.0"}`
`POST /api/calc`	all Plan-tab outputs, no side effects	inputs (§3.2 keys) → `{outputs, warnings, statements, arithmetic}` or `400 {errors: {field: message}}`
`POST /api/plans`	save a plan version	inputs → `{plan_id, version, folder, report_path}`

Method and path	Purpose	Request → Response
`GET /api/plans`	list for History and the Actuals dropdown	→ `[{plan_id, version, saved_at, system, period_label, verdict, n_headline, hours, warnings, actuals: {...} or null, unreadable: "error" or null}]`
`GET /api/plans/<plan_id>`	one plan, latest version, plus latest actuals	→ `{plan, actuals}`
`GET /api/prefill?system=`	§3.3	→ `{inputs, prefilled_from, p_source, actuals_recorded_at}` or `{}` if unknown system
`GET /api/systems`	auto-complete names	→ `["HR Helpdesk Bot", ...]`
`POST /api/actuals/calc`	Actuals results, no side effects	`{plan_id, reviewed, issues, agreement, notes}` → outputs and W8–W10
`POST /api/plans/<plan_id>/actuals`	record actuals version	same → `{version, report_path}`
`GET /api/log/rebuild`	rebuild CSV	→ `{path, rows}`
`POST /api/open-report`, `POST /api/open-folder`	`os.startfile` on the plan's report / folder	`{plan_id}` → `{ok: true}` or `{ok: false, path}`
`GET /api/method`	constants, formulas, integrity status	→ `{constants, formulas, integrity_check}`

Method and path	Purpose	Request → Response
`POST /api/quit`	shut down	→ `{ok: true}`, then the process exits

All error responses are JSON `{"error": "<short reason>", "details": ...}`; the UI shows the reason, never a stack trace. Full tracebacks go to the console.

---

10. Error handling strategy

#	Situation	Behaviour
E1	Invalid or missing input	`400` with a per-field message; the field is outlined and the message shown beneath it; outputs greyed; Save/Record disabled. Nothing is clamped.
E2	Calculation edge cases	Exactly as §4.8. No formula may raise: guard `N = 1`, `n ≥ N`, `n_max = 0`, `issues ∈ {0, reviewed}` explicitly.
E3	Workspace missing / not writable / disk full	Missing → created. Not writable or write failure → red global banner with the OS error text and the path; Save/Record disabled until a retry succeeds (`GET /api/health` reports `workspace_ok`). Temp file removed on failure; the previous files are untouched.
E4	Corrupt or unreadable plan/actuals JSON	History row marked unreadable with the error; excluded from prefill lookups, auto-complete and the log; console line. Never deleted or rewritten.
E5	Folder-name collision	`(2)`, `(3)` suffix rule in §3.1.
E6	Port in use	§2.3 item 3.
E7	Config unreadable	§8.6: message and exit 2.

#	Situation	Behaviour
E8	Startup integrity failure	§7.5: red banner, saving disabled, details on Method tab.
E9	Opening report/folder fails	Yellow toast showing the full path so it can be opened by hand.
E10	Unexpected exception in any endpoint	`500 {"error": "Unexpected error", "details": "<type>"}`; traceback to console; page stays usable.

---

11. Testing plan

Framework: `pytest`. File: `test_hitl_planner.py`, importing `hitl_planner` as a module. Persistence tests use pytest's `tmp_path` as the workspace (monkeypatch the config loader). Endpoint tests use Flask's test client. Run with `python -m pytest -q`; the calculation subset also runs via `python hitl_planner.py --selftest` and at every startup.

11.1 Answer key (the contract — every case is a test asserting the exact value)

Constants: 1.645 / 1.960 / 2.576. Decimals in formulas.

#	Case	Inputs	Expected
K1	Cochran + FPC	N = 1,000,000; C = 95 %; E = ±5 % absolute; p = 50 %	`n_cochran = 385` (raw 384.013)
K2	Cochran + FPC	N = 5,000; C = 95 %; E = ±5 %; p = 50 %	`n_cochran = 357` (raw 356.82)
K3	Cochran + FPC — **corrected from Riya's 1,844**	N = 1,000,000; C = 99 %; E = ±3 %; p = 50 %	`n_cochran = 1,840` (raw 1839.88; 1,844 is the uncorrected n0 rounded up)

#	Case	Inputs	Expected
K4	Assurance	C = 95 %; T = 1 %	`n_assure = 299` (raw 298.07)
K5	Detection power	n = 200; T = 1 %	`P = 0.8660` → displays 86.6 %
K6	Relative margin	N = 20,000; C = 95 %; p = 4 %; r = 25 % (E = 1 %)	`n_cochran = 1,374` (raw 1373.91)
K7	Reverse mode	N = 5,000; n = 357; p = 50 %; C = 95 %	`E_at = 0.04999` (assert to 4 dp: 0.0500) → displays 5.0 %
K8	Certify-backwards	n = 120; C = 95 %	`T_at = 0.02466` (assert to 4 dp: 0.0247) → displays 2.5 %
K9	Capacity-max	S = 3, H = 8, M = 12, s = 15 %, F = 50 (capacity 24 h); N = 50,000; p = 50 %; C = 95 %	`n_max = 70` (70 + 50 double = 120 reviews = exactly 24.0 h, FITS at n = 70, not at 71); `E_at(70) = 0.1171` → 11.7 %; `T_at(70) = 0.0419` → 4.2 %
K10	Double-review cap	s = 15 %, F = 50	`DR(40) = 40` ; `DR(400) = 60`
K11	Certify infeasible	C = 95 %; T = 0.1 %; N = 2,000	`n_assure = 2,995` (raw 2994.23); `n_assure > N` → W7 fires, `binding = "cochran"`

If any case fails, the developer MUST fix the code, never the expected value, and MUST stop and report rather than adjust a formula or constant.

11.2 Rule and boundary tests

- `ceil_safe` : `ceil_safe(356.0000000004) == 356` , `ceil_safe(356.0000001) == 357` .

- F5 binding: certify on with `n_assure > n_cochran` → assurance; equal → cochran; certify off → cochran regardless.
- F6: `DR ≤ n` always; `s = 0, F = 0` → `DR = 0`.
- F7: exact-equality case (K9) is FITS; 120.001 reviews-worth is DOES NOT FIT.
- F9: `n ≥ N` → 0; `N = 1` → 0.
- Each warning W1–W10: one test just inside the trigger, one just outside (e.g. W1 at E = 0.100 no, 0.1001 yes; W5 at N = 199 yes, 200 no; W6 at n = 0.8 N no, above yes; W10 at 0.80 no, 0.799 yes; W9 at exactly 2 × p and 0.5 × p not firing).
- Sensitivity grid: nine cells; margins switch with the mode; highlight logic; assurance mark when binding.
- Statements: seven entries always; S1/S2 variants selected correctly; numbers formatted per §4.7 (e.g. `0.25 %` for 0.0025).
- Validation: every limit in §3.2 refused at the boundary with a field-level message; nothing clamped.
- Rounding/display: hours 1 dp, rates < 1 % at 2 dp, thousands separators.

11.3 Persistence tests (tmp workspace)

- Save → `plan_v1.json` exists; reload through `/api/plans/<id>` → outputs identical to `/api/calc` for the same inputs.
- Save again → `plan_v2.json` exists; `plan_v1.json` byte-for-byte unchanged.
- `report.html` exists after save and after actuals; contains the plan_id, the seven statements and (after actuals) the "Actuals" section; no `<script>` tag and no external URL.
- Actuals → `actuals_v1.json`; record again → `actuals_v2.json`; v1 unchanged.
- Atomic write: simulate failure mid-write → no `.tmp` left behind, previous files intact, error reported.
- Log: three folders (one with actuals) → three rows, correct columns and order, `utf-8-sig` BOM present, notes with commas/newlines round-trip through `csv`.
- Corrupt JSON in one folder → History marks it unreadable; log has N−1 rows; prefill ignores it.

- Sanitising: `A/B: "C"` → `AB C`; reserved name `CON` → `CON_`; collision → `(2)` suffix.
- Prefill: existing system → last plan's inputs minus N, p from last observed rate floored at 0.1 %; system with plans but no actuals → last planned p; unknown system → `{}`; matching is case-insensitive.

11.4 Endpoint tests (Flask test client)

Every endpoint in §9: success shape, `400` on invalid input with `errors` map, `404` on unknown plan_id, `500` shape on a forced exception (page does not crash). `/api/health` includes `workspace_ok`.

11.5 Startup

- `--selftest` exits 0 with 11 PASS lines; with a deliberately broken constant (monkeypatched) exits 1 and names the failing cases.
- Integrity failure sets the flag that disables Save/Record (assert via `/api/method`).
- Config: missing → created with defaults; broken → exit code 2.

11.6 Manual acceptance checklist (TJ, in the browser, no code reading)

1. Type K1–K4 into the Plan tab (N as given; certify on or off gives the same result for K1–K3) and confirm 385, 357, 1,840, and 299 on the assurance card.
2. Scenario: N = 50,000, 95 %, ±5 % absolute, p 50 %, 3 SMEs × 8 h × 12 min, 15 % / 50, certify on at 1 %. Expect: headline **382 conversations = 88.0 SME hours**, DOES NOT FIT (capacity 24.0 h), 58 double reviews, binding = precision, detection ≈ 97.8 %, W2 showing. Trade-off panel: capacity allows **70**, buys **±11.7 %**, certify below **4.2 %**.
3. Change N to 1,000,000: headline becomes 385 conversations, 58 double, 88.6 h.
4. With certify still on, push the slider to ±15 %: n stops at **299** and the read-out says the certify claim binds (still DOES NOT FIT: 299 + 50 double = 69.8 h). Turn certify off: n drops to **43**, DR = 43, 86 reviews, **17.2 h → FITS**. Press Adopt: the badge turns green, the panel closes, W1 and W4 are showing.
5. Save; open the folder; confirm `plan_v1.json` contains every input typed; open `report.html` and print to PDF.

6. Record actuals (reviewed 382, issues 11, agreement 86 %): observed rate 2.9 %, achieved margin ≈ ±1.7 %, W9 showing; `actuals_v1.json` and the Actuals section in the report exist; `plans_log.csv` opens in Excel with the row filled.
7. Start a new plan for the same system next month: banner says prefilled, p shows 2.9 % with source "last observed", N is blank.
8. Method tab shows "Answer key: 11 / 11 passed" and the live arithmetic matches the headline.
9. Quit button stops the server; running the script again reopens the browser on the same port.

---

12. CLAUDE.md non-negotiables (paste verbatim into `CLAUDE.md`)

```
# HITL Sample Size Planner — non-negotiables (spec v1.0, method
1.0)

1. One file: hitl_planner.py. Tests in test_hitl_planner.py. No
other modules, no packages.
2. Flask only. Standard library for everything else. No other
pip installs without asking.
3. Bind to 127.0.0.1 only. No outbound HTTP, no CDN assets, no
telemetry. Must work offline.
4. All statistics are computed in Python. The browser only sends
inputs and paints results.
5. Formulas F1–F10 and the constants Z = 1.645 / 1.960 / 2.576
are exactly as specified.
   Never substitute another sample-size method, never use "more
precise" z-values,
   never change rounding (every n rounds UP via ceil_safe).
6. THE ANSWER KEY (spec §11.1, 11 cases) IS THE CONTRACT. A
failing test means the code is
   wrong. Never edit an expected value. Stop and report instead
of adjusting a formula.
7. Saved plan and actuals JSON files are immutable: never opened
for writing. New versions only.
   plans_log.csv and report.html are generated from the JSON;
never appended, never hand-edited.
8. All file writes are atomic (temp file + os.replace).
9. No inputs, outputs, warnings, statements, tabs or files
```

```
beyond the spec without asking.
10. No deletion of user data from the UI or the code.
11. One blueprint step per session. Run the tests after every
step. Commit only when green.
```

---

13. Build order (guidance for the blueprint; each line becomes one or more `/step` prompts)

1. Skeleton: constants, `ceil_safe`, config load/create, Flask app with `/`, `/api/health`, `/api/quit`, port check, browser open, `--selftest` stub.
2. Calculation core F1–F4, F8–F10 + answer-key tests K1–K8, K11 (pure functions, no Flask).
3. Headline rule F5, DR F6, hours/verdict F7 + tests K9, K10 and §11.2 rule tests.
4. Trade-off search (`n_max`, margin and threshold at capacity) + tests.
5. Sensitivity grid + tests.
6. Warnings W1–W7 + boundary tests.
7. Statements S1–S7 and the arithmetic trace + tests.
8. Validation and `POST /api/calc` + endpoint tests.
9. Plan tab UI: inputs, live calc, headline, trade-off panel with slider and Adopt, grid, assurance card, statements, warnings.
10. Persistence: save versions, sanitising, atomic writes, `report.html` (plan sections) + tests.
11. History tab, `/api/plans`, `/api/systems`, prefill + tests.
12. Actuals calculations (§4.6), W8–W10, Actuals tab, `actuals_vN.json`, report Actuals section + tests.
13. `plans_log.csv` generation + tests.
14. Method tab, startup integrity check, `--selftest` completion.
15. Final pass: README, acceptance checklist §11.6 run by TJ, tag v1.0.0.

Appendix A — Interview decision log (27 Sep 2026)

Q	Decision
1	Personal desktop tool, Python, TJ's use only
2	One file, Flask, all maths in Python, tests as pytest; small JS layer for live updates
3	Workspace folder: one sub-folder per plan plus a generated `plans_log.csv`
4	Period per plan: Month (default) / Week, no custom ranges; several systems supported from day one via a free-text field with auto-complete; identity = date-first `period - system`
5	Plan + Actuals: reviewed, issues, agreement %, notes; observed rate feeds the next plan's p
6	Relative-margin switch (E = r × p) plus W3 when the absolute margin is as large as p
7	Statistical conventions get defaults; N, SMEs, hours, minutes are blank and required on a system's first plan; later plans prefill from history, N excepted
8	Uniform team capacity; DR = larger of floor and share, capped at n; each DR costs a full second review; FITS when (n + DR) × min ÷ 60 ≤ capacity
9	Certify toggle, default on: headline = larger of Cochran and assurance, binding requirement named; detection power under the assurance card either way
10	Trade-off panel: capacity-max n, achievable margin, certify-backwards threshold; slider is what-if only, "Adopt this margin" commits; confidence not on the slider
11	3 × 3 sensitivity grid (margins × confidence), headline rule inside the grid
12	Warnings W1–W10 as listed, W7 red
13	Seven fixed statements
14	Method tab: formulas with rounded constants, assumptions, live arithmetic, references, method version

Q	Decision
15	Save writes immutable `plan_vN.json` + standalone `report.html`; PDF by browser print; actuals in their own file
16	Save is deliberate; `plans_log.csv` regenerated from the folders; column set; hyphenated, sanitised folder names
17	Four tabs: Plan, Actuals, History, Method; desktop only; Pareto/IMR look
18	`python hitl_planner.py`, 127.0.0.1, fixed port, single instance, Quit button, `config.json`, Flask only, Python 3.11, no exe in v1
19	Rounding (always up), display precision, input limits (refuse, never clamp)
20	Answer key: K3 corrected to 1,840; constants exact; K6–K11 added
21	Test suite as §11; startup integrity check; CLAUDE.md non-negotiables
22	v2 list; no delete in the UI; actuals versioned like plans

Appendix B — Departures from Riya's outline, and defaults assumed without an explicit question

Departures from the outline (all confirmed in the interview):

- Python/Flask desktop tool instead of plain HTML/JS; persistence added (the outline had "no data stored anywhere").
- Answer-key case 3 is **1,840**, not 1,844 — the outline's value ignored the population correction it specified.
- Double-review count capped at n (the outline would demand 50 double reviews of a 40-item sample).
- Certify toggle with the headline taking the larger of Cochran and assurance (the outline kept them separate).
- Relative-margin mode; Actuals tab; prefill from history; sensitivity widened to 3 × 3; seven statements instead of three; ten warnings instead of four.
- N has no default (the outline's 50,000 would produce a saved plan with an invented population).
- Tests live in a second file — a deliberate, minimal departure from the strict single-file rule; the tool itself remains one file.

Defaults assumed by the spec author, open to change without a method-version bump:

- Port 5062 (to be verified against Pareto/IMR before the first commit); `config.json` layout.
- Floating-point tolerance 1e-9 in `fits()` and `ceil_safe` rounding at 9 decimals.
- Slider steps: 0.5 % absolute, 1 % relative; trade-off panel shown only when DOES NOT FIT.
- Period picker defaults to the current month / ISO week.
- "Most recent" plan/actuals determined by `saved_at` / `recorded_at`, not by period label.
- Next-cycle p floored at 0.1 % when zero issues were observed.
- Log encoding `utf-8-sig`; log sort order; CSV column order.
- System matching case-insensitive; folder collision suffix `(2)`.
- `--selftest` flag; JSON error shape; `os.startfile` for opening reports and folders.
- Detection power shown even when certify is off; S2 "certify off" wording.

Appendix C — Glossary (plain English)

- **Population (N)** — every conversation the AI produced in the period.
- **Sample (n)** — the conversations the SMEs actually read.
- **Margin of error (E)** — how far the measured rate may be from the true rate. Absolute: ±5 percentage points. Relative: ±25 % of the rate itself.
- **Confidence (C)** — how often, over many repetitions, the true rate would fall inside the margin. 95 % is the convention.
- **Expected issue rate (p)** — the best guess before reviewing. 50 % is the worst case and gives the largest sample.
- **Finite population correction** — the adjustment that lets a smaller sample do the job when the population is small.
- **Assurance / certify claim** — "we reviewed n_assure conversations, found no issue, so the rate is below T". Holds only if none is found.
- **Detection power** — the chance a sample contains at least one issue if the true rate is T.
- **Double review** — a second SME independently reviews a subset so reviewer agreement can be measured.

- **Capacity** — SMEs × net hours available this period.
- **Binding requirement** — whichever of precision and certification demands the larger sample.